Reviewer Pack — Real Radical Rank Lower Bounds SOS Degree for Max-CUT

Entry e37540873b70 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-11 13:30:33 UTC

Real Radical Rank Lower Bounds SOS Degree for Max-CUT

Entry ID: `e37540873b70` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-11 13:30:33 UTC

1. Conjecture

Field A (mathematical branch): Real Algebraic Geometry (Real Radical Ideals) **Field B** (complexity object): SOS Degree for Max-CUT Instances

Statement:

> For any Max-CUT instance with n variables, the SOS degree required to achieve Ω(1) approximation is at least the real radical rank of the ideal defining the instance. Formally, if I is the ideal generated by the quadratic forms of the graph, then deg_SOS(I) ≥ rank_real_radical(I).

Rationale (proposer's reasoning):

> Real radical ideals capture the algebraic structure of real solutions to polynomial systems. By linking their rank to SOS degree, we expose a geometric constraint on the complexity of semidefinite relaxations, potentially revealing inherent limitations in approximation algorithms.

Taxonomy category: `BOUNDED_ARITHMETIC` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `f7afac124580749b`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.95	SAFE	SAFE
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM_1	LLM_2
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=0, elapsed=19.1s

5.1 Generated Python source

```python
import random
import math
from itertools import combinations

def generate_random_graph(n):
    edges = set()
    for i in range(n):
        for j in range(i + 1, n):
            if random.choice([True, False]):
                edges.add((i, j))
    return edges

def quadratic_forms_from_edges(edges, n):
    forms = []
    for i in range(n):
        form = [0] * n
        form[i] = 1
        for j in range(i + 1, n):
            if (i, j) in edges or (j, i) in edges:
                form[j] = -1
        forms.append(form)
    return forms

def matrix_multiplication(A, B):
    m, k = len(A), len(B[0])
    result = [[0] * k for _ in range(m)]
    for i in range(m):
        for j in range(k):
            for l in range(len(B)):
                result[i][j] += A[i][l] * B[l][j]
    return result

def gaussian_elimination(A, b):
    n = len(A)
    augmented_matrix = [A[i] + [b[i]] for i in range(n)]
    for i in range(n):
        max_row = i
        for j in range(i + 1, n):
            if abs(augmented_matrix[j][i]) > abs(augmented_matrix[max_row][i]):
                max_row = j
```

```
        augmented_matrix[i], augmented_matrix[max_row] = augmented_matrix[max_row], augmented_matr
        pivot = augmented_matrix[i][i]
        for j in range(i, n + 1):
            augmented_matrix[i][j] /= pivot
        for j in range(n):
            if j != i:
                factor = augmented_matrix[j][i]
                for k in range(i, n + 1):
                    augmented_matrix[j][k] -= factor * augmented_matrix[i][k]
    return [row[-1] for row in augmented_matrix]

def real_radical_rank(I):
    forms = quadratic_forms_from_edges(I, len(I))
    A = []
    for form in forms:
        A.append(form)
    b = [0] * len(forms)
    solution = gaussian_elimination(A, b)
    rank = sum(1 for x in solution if abs(x) > 1e-6)
    return rank

def sos_degree(I):
    n = len(I)
    forms = quadratic_forms_from_edges(I, n)
    A = []
    for form in forms:
        A.append(form)
    b = [0] * len(forms)
    solution = gaussian_elimination(A, b)
    degree = sum(1 for x in solution if abs(x) > 1e-6)
    return degree

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.choice([5, 10, 15, 20, 30, 40])
    I = generate_random_graph(n)
    rank_real_radical_I = real_radical_rank(I)
    sos_degree_I = sos_degree(I)
    return {
        "metric_name": "sos_degree",
        "metric_value": sos_degree_I,
        "instances_tested": 1,
        "conjecture_holds": sos_degree_I >= rank_real_radical_I,
        "counterexample": "" if sos_degree_I >= rank_real_radical_I else f"Graph with {n} variable
    }

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2**i - 1 for i in range(5, 35)]
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
```

```
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"Graph with {n} variables and {len(I)} edges\" f
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
re_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'sos_degree', 'metric_value': 0, 'instances_tested': 1, 'conjecture_holds': T
TRIAL: {'metric_name': 'sos_degree', 'metric_value': 0, 'instances_tested': 1, 'conjecture_holds': T
TRIAL: {'metric_name': 'sos_degree', 'metric_value': 0, 'instances_tested': 1, 'conjecture_holds': T
TRIAL: {'metric_name': 'sos_degree', 'metric_value': 0, 'instances_tested': 1, 'conjecture_holds': T
TRIAL: {'metric_name': 'sos_degree', 'metric_value': 0, 'instances_tested': 1, 'conjecture_holds': T
TRIAL: {'metric_name': 'sos_degree', 'metric_value': 0, 'instances_tested': 1, 'conjecture_holds': T
TRIAL: {'metric_name': 'sos_degree', 'metric_value': 0, 'instances_tested': 1, 'conjecture_holds': T
TRIAL: {'metric_name': 'sos_degree', 'metric_value': 0, 'instances_tested': 1, 'conjecture_holds': T
TRIAL: {'metric_name': 'sos_degree', 'metric_value': 0, 'instances_tested': 1, 'conjecture_holds': T
TRIAL: {'metric_name': 'sos_degree', 'metric_value': 0, 'instances_tested': 1, 'conjecture_holds': T
TRIAL: {'metric_name': 'sos_degree', 'metric_value': 0, 'instances_tested': 1, 'conjecture_holds': T
TRIAL: {'metric_name': 'sos_degree', 'metric_value': 0, 'instances_tested': 1, 'conjecture_holds': T
TRIAL: {'metric_name': 'sos_degree', 'metric_value': 0, 'instances_tested': 1, 'conjecture_holds': T
TRIAL: {'metric_name': 'sos_degree', 'metric_value': 0, 'instances_tested': 1, 'conjecture_holds': T
TRIAL: {'metric_name': 'sos_degree', 'metric_value': 0, 'instances_tested': 1, 'conjecture_holds': T
RESULT: SUPPORTED mean=0.0 std=0.0 support_fraction=1.0
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> n is not specified but likely ≤1 (instances_tested=1 per trial), making SOS degree trivially 0. Metric saturation occurs if radical rank is always 0 for these instances, rendering the bound vacuous. The test lacks adversarial cases with non-zero radical rank.

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test shows 100% support but uses trivial instances (n ≤ 1) where SOS degree is zero, making the bound vacuous. The critic correctly challenges the lack of adversarial cases with non-zero radical rank. | next: Test with larger n and non-trivial radical rank examples to validate the conjecture's generality

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	109465
2	propose	ollama_remote	qwen3:8b	0	0	106201
3	propose	ollama_remote	qwen3:8b	0	0	72358
4	preregistration	ollama_remote	qwen3:8b	0	0	24181
5	novelty	ollama_remote	qwen3:8b	0	0	20654
6	novelty	ollama_remote	qwen3:8b	0	0	14658
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	34390
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11912
9	critic	ollama_remote	qwen3:8b	0	0	26655
10	judge	ollama_remote	qwen3:8b	0	0	19067

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 439542 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/e37540873b70.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/e37540873b70.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/e37540873b70.tar.gz` (if generated)